

Unipile Authentication & Subscription Management

Project Beacon · G3 Recruitment Automation Platform

Document Scope: Complete Connection Setup, Recruiter & Owner Flows, Edge Cases

1. How to Connect to Unipile (Environment & One-Time Setup)

1.1 Environment Variables (Server-Side Only)

UNIPILE_DSN="api8.unipile.com:13081"	# Tenant DSN (separate for Staging vs Prod)
UNIPILE_API_KEY="usr_live_xxxx..."	# X-API-KEY header (never expose to frontend)
UNIPILE_WEBHOOK_SECRET="whsec_xxxx..."	# Custom shared secret for webhook verification
UNIPILE_WEBHOOK_PATH_TOKEN="tok_32bytes"	# Unguessable URL segment:
/api/unipile/webhook/\$token	
APP_BASE_URL="https://app.g3.example"	# Public origin for callbacks & redirects

1.2 One-Time Webhook Registration (Run Once at Deploy)

POST https://api8.unipile.com:13081/api/v1/webhooks
Headers: { "X-API-KEY": "<key>", "Content-Type": "application/json" }
Body: {
"request_url": "https://app.g3.example/api/unipile/webhook/tok_32bytes",
"source": "account_status",
"name": "g3-account-status",
"format": "json",
"enabled": true,
"headers": [
{ "key": "Content-Type", "value": "application/json" },
{ "key": "X-G3-Webhook-Secret", "value": "<UNIPILE_WEBHOOK_SECRET>" }
]
}

Note on webhook authenticity: Unipile's marketing site claims webhooks are signed with a verifiable signature header, but the OpenAPI schema for `POST /webhooks` documents no such field — no secret, no signature, no algorithm. Until Unipile support confirms which is current, we do **not** rely on a vendor-provided signature. The `X-G3-Webhook-Secret` custom header above (compared with a timing-safe check) plus the unguessable `UNIPILE_WEBHOOK_PATH_TOKEN` path segment are our own substitute defenses, not a documented Unipile capability.

2. Detailed Recruiter Flow — Adding a New Account

Which Actions Redirect to Unipile vs Stay In-App?

Action	Where It Happens	Redirect to Unipile?
--------	------------------	----------------------

Connect New Account	Recruiter clicks button in G3 Settings	Yes — Full-page redirect to Unipile's hosted wizard
Reconnect Broken Account	Recruiter clicks "Reconnect" on red badge	Yes — Full-page redirect (type: "reconnect")
Disconnect Own Account	Recruiter clicks "Disconnect" on green badge	No — G3 backend calls Unipile API silently
Owner Revokes Account	Sundar clicks "Revoke" in Owner Dashboard	No — G3 backend calls Unipile API silently

2.1 Adding a New Account (Step-by-Step)

Recruiter clicks "Connect" --> Server checks seats & mints link --> REDIRECT to Unipile Wizard

Badge: Green "Connected" <-- Webhook: SYNC_SUCCESS <-- Webhook: CREATION_SUCCESS (Blue Badge)

- 1. Recruiter Action:** Clicks "Connect LinkedIn" in G3 Settings.
- 2. Server Pre-Checks:**
 - Verifies recruiter is authenticated (session cookie).
 - Checks seat capacity: counts `ConnectedAccount` rows where `status != DISCONNECTED` against `SystemConfig['unipile.max_slots']`. If full, returns error: *"Seat limit reached. Contact your admin."*
 - Checks for existing connection: if a `ConnectedAccount` already exists for this `userId` + `provider` and is not `DISCONNECTED`, returns error: *"You already have a LinkedIn account connected."*
 - Checks for in-flight attempts: if an unexpired `UnipileAuthAttempt` exists for this user, returns error: *"A connection attempt is already in progress."*
- 3. Mint Auth Link:** Creates a `UnipileAuthAttempt` row (15-min TTL nonce) and calls Unipile:

```
POST /api/v1/hosted/accounts/link
{
  "type": "create",
  "providers": ["LINKEDIN"],
  "api_url": "https://api8.unipile.com:13081",
  "expiresOn": "2026-07-31T12:15:00.000Z",
  "name": "<single_use_nonce>",
  "notify_url": "https://app.g3.example/api/unipile/webhook/tok_32bytes",
  "success_redirect_url": "https://app.g3.example/recruiter/settings?connect=ok",
  "failure_redirect_url": "https://app.g3.example/recruiter/settings?connect=failed",
  "bypass_success_screen": true,
  "single_use": true,
  "disabled_options": ["cookie_auth", "proxy", "autoproxy"],
}
```

```
"sync_limit": { "MESSAGING": { "chats": 90, "messages": 500 } }
}
```

4. **Redirect:** G3 frontend performs `window.location.assign(url)` — full-page redirect, never an iframe.
5. **Recruiter Authenticates:** Enters password, 2FA, QR code on Unipile's hosted page. G3 never sees any of this.
6. **Stage 1 — notify_url fires (CONNECTING 🟦):**
 - Unipile POSTs flat payload: `{ "status": "CREATION_SUCCESS", "account_id": "acct_123", "name": "<nonce>" }`
 - G3 resolves nonce → `userId`, creates `ConnectedAccount` (status: `CONNECTING`), burns the nonce.
 - Returns `200 OK` promptly. *(The 200-within-30-seconds / 5-retry policy is explicitly documented for registered account_status webhooks; we apply the same discipline to notify_url as a safe default, though Unipile's docs don't state its retry behavior for this specific callback — worth confirming with support.)*
7. **Browser Returns:** Unipile redirects recruiter back to `success_redirect_url` . UI shows **Blue Badge ("Connecting... history is downloading")**.
8. **Stage 2 — account_status webhook fires (OK 🟢):**
 - Unipile POSTs nested payload: `{ "AccountStatus": { "account_id": "acct_123", "message": "SYNC_SUCCESS" } }`
 - G3 updates status to `OK` , sets `lastSyncedAt` . Badge flips to **Green ("Connected")**.

2.2 Reconnecting a Broken Account (Step-by-Step)

Badge turns Red (CREDENTIALS) --> Recruiter clicks "Reconnect" --> REDIRECT to Unipile Wizard

Badge: Green "Connected" again <-- Webhook: RECONNECTED <----- Re-authenticates

1. **Trigger:** Unipile fires `CREDENTIALS` (password changed, 2FA expired, LinkedIn checkpoint, or token expiry after 1 year for LinkedIn/Instagram). G3 sets status to `RECONNECTION_NEEDED` 🟥.
2. **Recruiter Clicks "Reconnect":** G3 backend mints a reconnect link:

```
POST /api/v1/hosted/accounts/link
{ "type": "reconnect", "reconnect_account": "acct_123", ... }
```

3. **Redirect:** Same full-page redirect to Unipile wizard. Recruiter re-enters credentials.
4. **Webhook fires:** `RECONNECTED` → status back to `OK` 🟢. The `AccountDegradation` window is closed (`endedAt = now`). All messages received during the disconnection period are back-filled automatically by Unipile.

2.3 Disconnecting an Account (Recruiter-Initiated) — NO REDIRECT

Recruiter clicks "Disconnect" --> G3 Server calls `DELETE /api/v1/accounts/{id}` --> DB: DISCONNECTED

- 1. **Recruiter Clicks "Disconnect"**: A confirmation dialog appears: "Are you sure? This will stop all message sync and outreach for this account."
- 2. **Server-Side Only**: G3 backend calls Unipile's `DELETE /api/v1/accounts/{accountId}` . **No redirect occurs**. The recruiter stays on the same settings page.
- 3. **DB Update**: `ConnectedAccount` status → `DISCONNECTED` . Unipile also fires a `deleted` webhook as confirmation.
- 4. **UI Update**: Badge changes to **Gray ("Not Connected")** with a fresh "Connect" button.

Verification note: the exact method/path (`DELETE /api/v1/accounts/{id}`) is inferred from Unipile's reference index entry "Delete an account" (`accountscontroller_deleteaccount`) and standard REST convention — the live reference page's full request/response shape was not directly fetched and should be confirmed before this ships.

3. Owner Flow for Sundar / Ethan — Seat & Subscription Management

3.1 Setting the Seat Cap

- 1. Sundar opens Owner Dashboard (`client/src/routes/owner.settings.tsx`).
- 2. Clicks "Edit Seat Cap" and sets `unipile.max_slots` (e.g. 10).
- 3. This value is stored in `SystemConfig` and checked server-side before every auth link is minted.

3.2 Revoking a Recruiter's Account (Owner-Initiated) — NO REDIRECT

Sundar clicks "Revoke" --> G3 Server calls `DELETE /api/v1/accounts/{id}` --> DB: `DISCONNECTED`

- 1. **Sundar Clicks "Revoke"**: A confirmation dialog appears: "This will disconnect Sarah Connor's LinkedIn account. She will need to reconnect manually."
- 2. **Server-Side Only**: G3 backend calls `DELETE /api/v1/accounts/{accountId}` . **No redirect**. **The owner stays on the dashboard**.
- 3. **DB Update**: `ConnectedAccount` status → `DISCONNECTED` . Seat count decreases.
- 4. **Recruiter Notification**: An in-app alert and email are sent to the affected recruiter.
- 5. **Billing Note**: The dashboard reminds Sundar: "Disconnecting frees capacity for future 30-day billing periods. It does not reduce the current period's peak."

3.3 Dashboard Visibility

OWNER DASHBOARD – UNIPILE SEAT MANAGEMENT			
Active Seats: [8 / 10 Used]		=====> (80%)	Monthly Peak Est: €49.00/mo
Recruiter	Provider	Status	Actions
Sarah Connor	LINKEDIN	● Connected	[Revoke]
Alex Mercer	LINKEDIN	● Connecting	[Revoke]
David Miller	GOOGLE	● Reconnect	[Send Reminder Email]

4. Edge Cases & Error Handling

4.1 Auth Link & Connection Edge Cases

Edge Case	What Happens	How G3 Handles It
Link expires (15-min TTL)	15 minutes is our own chosen expiresOn value, not a Unipile-imposed limit — Unipile lets the caller set any expiresOn timestamp. The one rule Unipile <i>does</i> enforce independent of that value: all links die on Unipile's daily server restart regardless of stated expiry. We pick a short TTL (15 min) deliberately, as a safety margin.	Show error on failure redirect: <i>"Link expired. Click Connect again."</i> Generate a fresh link on demand. Never email or cache links.
Seat cap reached mid-flow	A second recruiter fills the last slot while the first is still authenticating.	The webhook callback (notify_url) still fires and we still record the account. This is acceptable because the seat was available when the link was minted. To prevent this entirely, reserve the slot at link-mint time by creating the UnipileAuthAttempt row.
Recruiter tries to connect same provider twice	ConnectedAccount has a @@unique([userId, provider]) constraint.	Backend rejects with: <i>"You already have a LinkedIn account connected."</i> Prevents accidental double billing.
HTTP 425 — Auth already in progress	Unipile returns auth_in_progress if a connection attempt is already running for this account.	Surface: <i>"A connection is already in progress. Please wait or try again in a few minutes."</i>
Recruiter closes browser mid-auth	The recruiter navigates away from Unipile's wizard before completing.	The notify_url never fires. The UnipileAuthAttempt nonce expires after 15 minutes. No DB row is created. The recruiter can simply click Connect again.
CREATION_FAIL	The authentication itself failed on Unipile's side (wrong password, captcha fail).	G3 updates status to CONNECT_FAILED 🟡 (Amber Badge: <i>"Could not connect"</i>). Action: <i>"Try Again"</i> .

4.2 Webhook & Status Edge Cases

Edge Case	What Happens	How G3 Handles It
-----------	--------------	-------------------

Two different webhook payload shapes	notify_url sends flat { "status": "...", "account_id": "...", "name": "..." }. The registered account_status webhook sends nested { "AccountStatus": { "account_id": "...", "message": "..." } }.	A single handler reads both: body.AccountStatus?.message ?? body.status. This is the most critical implementation detail — handling only one shape means broken accounts never show as broken.
Duplicate SYNC_SUCCESS (LinkedIn Premium)	LinkedIn Premium accounts send two SYNC_SUCCESS payloads, distinguished by a product field.	All webhook writes are idempotent. The UnipileWebhookEvent.dedupeKey prevents double-processing. Setting status to OK twice is harmless.
Webhook retries (5x)	If G3 fails to return 200 within 30 seconds, Unipile retries up to 5 times with increasing delay between attempts . (<i>Unipile's developer docs describe this as "incremental time delay"; a separate marketing page describes it as "exponential backoff" — the two aren't confirmed to mean the same thing, so treat the exact backoff curve as unconfirmed and design the handler to be safe under either.</i>)	Handler must persist the raw event and return 200 fast. Heavy processing happens asynchronously after the response.
PERMISSIONS (OAuth scope revoked)	Google/Microsoft OAuth scopes were declined or revoked by the recruiter.	Status → PERMISSION_REVOKED 🚫. UI shows "Re-grant Access" — this needs a re-consent prompt, not a password prompt. Different copy from RECONNECTION_NEEDED.
Unknown/unmapped status string	Unipile adds a new status event in a future update.	normalizeUnipileStatus() has a default branch that maps unknown strings to SYNC_STOPPED and logs an error alert. The raw string is always persisted for debugging.

4.3 Owner & Billing Edge Cases

Edge Case	What Happens	How G3 Handles It
Owner deletes account while recruiter is mid-reconnect	Sundar revokes an account while the recruiter is on Unipile's wizard.	The recruiter's notify_url callback will fire with RECONNECTED, but G3 has already set status to DISCONNECTED. The handler checks current status and does not resurrect a DISCONNECTED account — the recruiter sees "Account was removed by your admin."

Recruiter connects 3 providers (LinkedIn + Gmail + WhatsApp)	Each provider is a separate billed account. One recruiter = 3 seats consumed.	The seat cap counts ConnectedAccount rows, not users. The UI clearly shows: <i>"Each connected channel uses one seat."</i>
Owner reduces seat cap below current usage	Sundar sets max_slots from 10 to 5, but 8 accounts are active.	Existing connections are not forcefully disconnected. No new connections are allowed until usage drops below the new cap. Dashboard warns: <i>"8 active seats exceed your cap of 5. No new connections until seats are freed."</i>
Test accounts on production DSN	Connecting test accounts on the production Unipile tenant inflates the 30-day peak invoice.	Use a separate Unipile DSN (UNIPILE_DSN) for staging. This is why separate environment configs exist.
LinkedIn Recruiter session conflict	LinkedIn Recruiter accounts enforce single active sessions. Connecting via Unipile may log the recruiter out of their browser session.	Pass disabled_features: ["linkedin_recruiter"] in the auth link to suppress Recruiter mode. Ask the recruiter before onboarding a Recruiter-seat user.

5. Database Footprint Summary

Model / Table	Source	Purpose
User	Existing	Role-based access (OWNER, RECRUITER, CONTRACTOR).
SystemConfig	Existing	Stores unipile.max_slots, rate limits.
ConnectedAccount	To Add	Maps userId ↔ unipileAccountId, provider, status, invite counters.
UnipileAuthAttempt	To Add	15-min TTL single-use nonces for user ↔ callback correlation.
UnipileWebhookEvent	To Add	Idempotency deduplication log (prevents double-processing).
AccountDegradation	To Add	Tracks non-OK windows to exclude from recruiter SLA metrics.

6. Phased Implementation

- **Phase 0:** Provision Staging & Prod Unipile DSNs; verify ?port= workaround; execute GDPR DPA.
- **Phase 1:** Database migration; Hosted Auth endpoints; webhook handler (both shapes); Recruiter & Owner Settings UIs.
- **Phase 2:** Messaging webhooks; echo filtering; inbound reply tracking.
- **Phase 3:** Rate Governor; circuit breaker; automated sequence pacing.